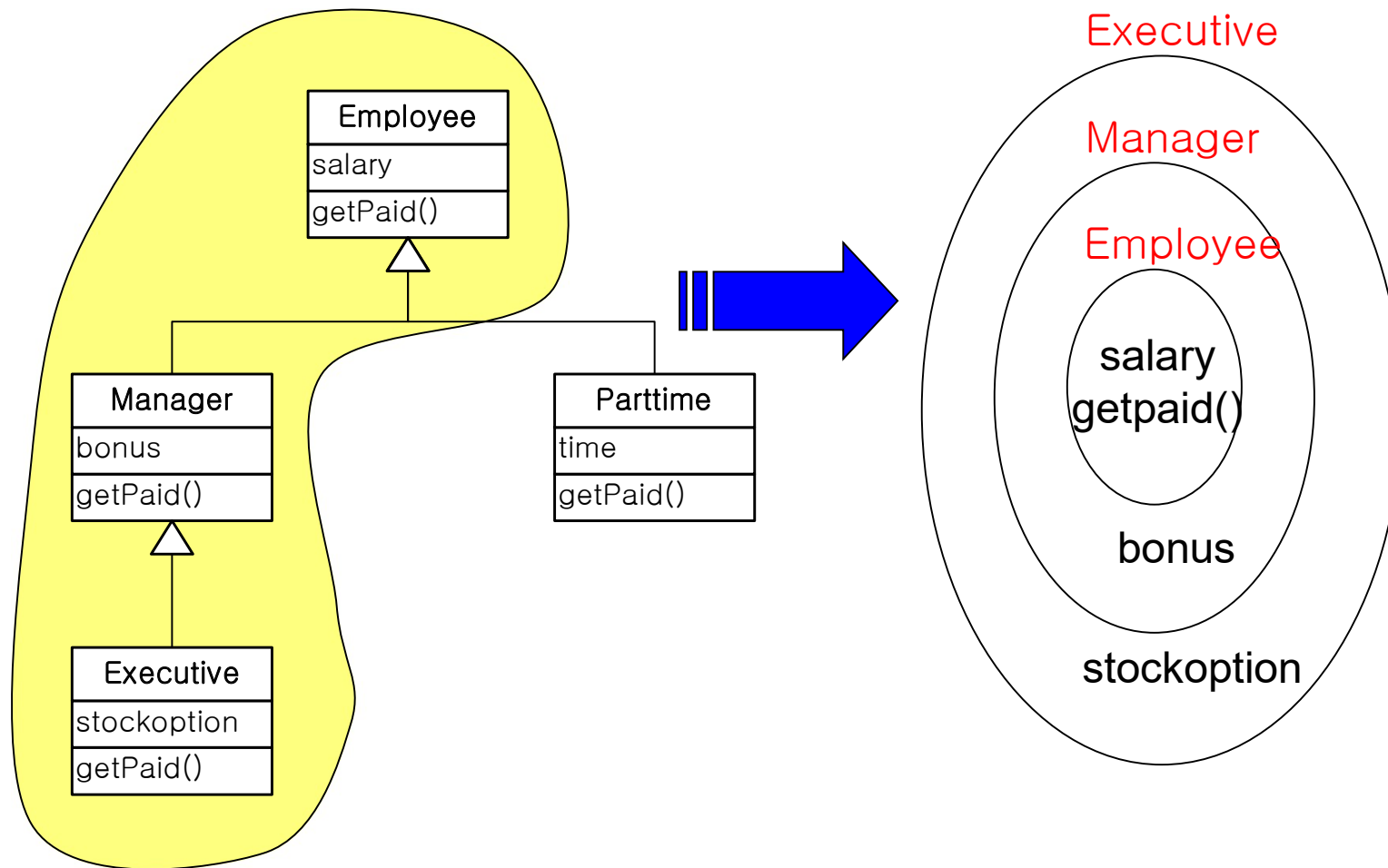




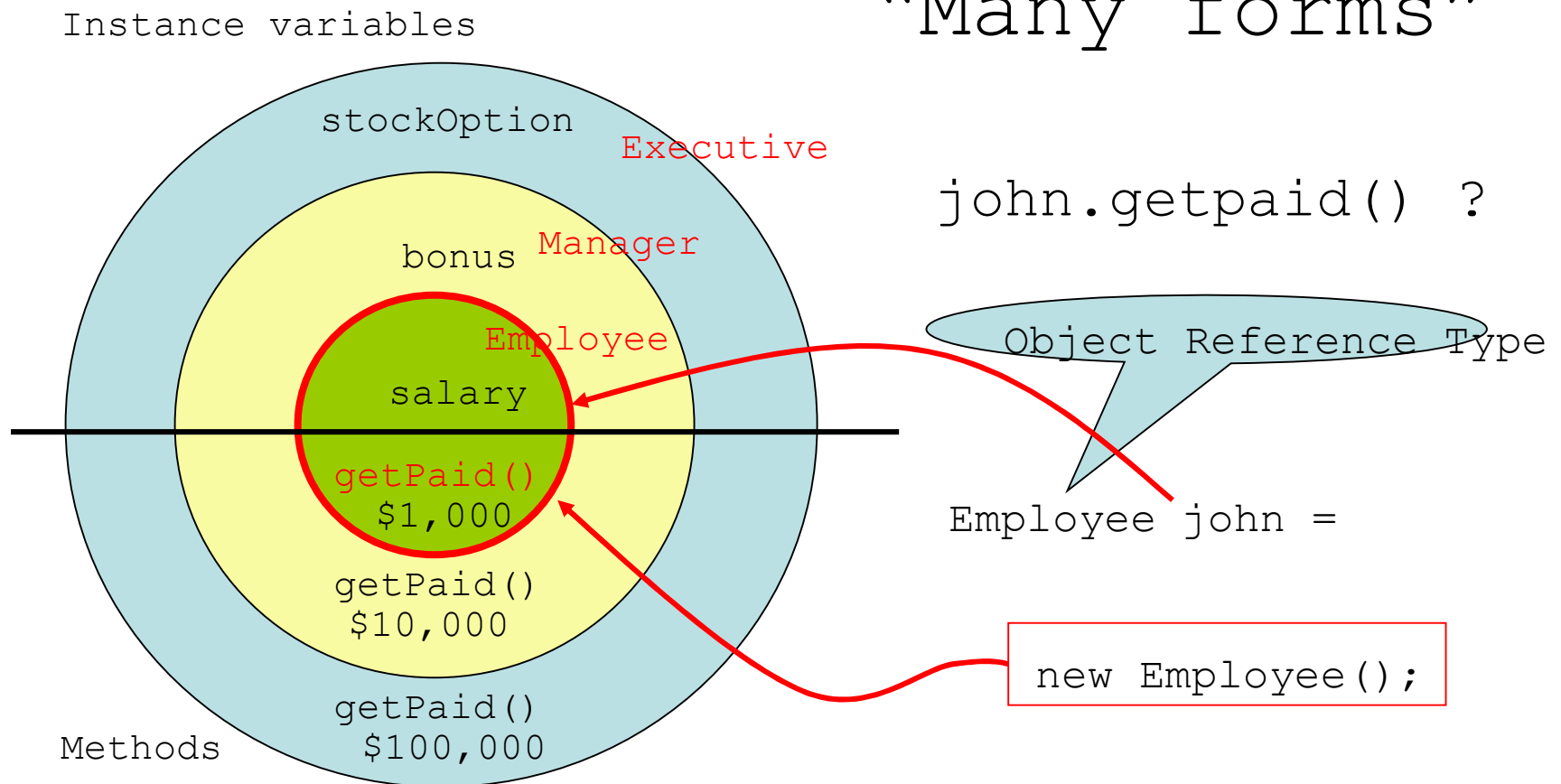
Polymorphism ("Many Forms")





Polymorphism

"Many forms"

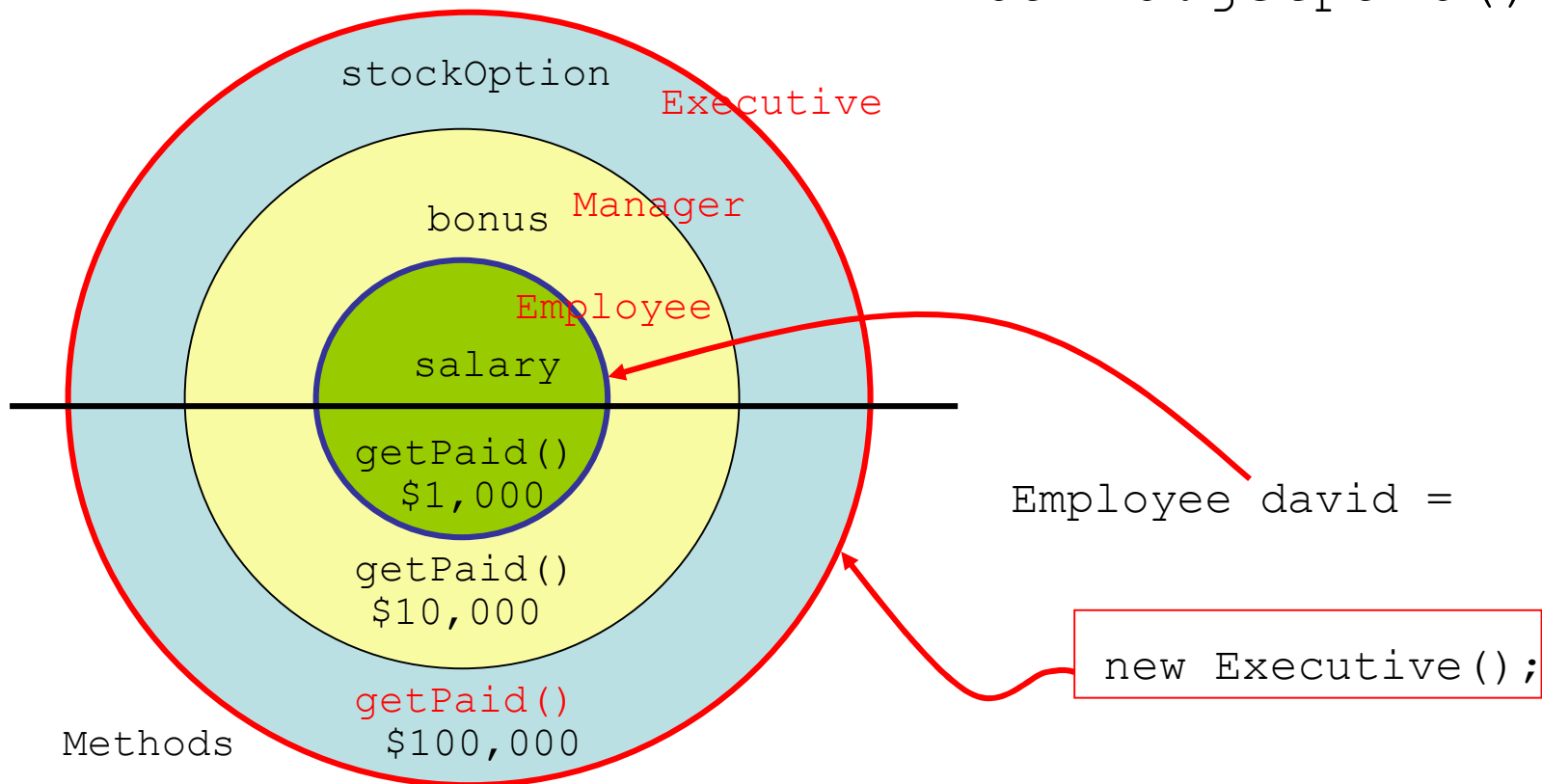




Polymorphism

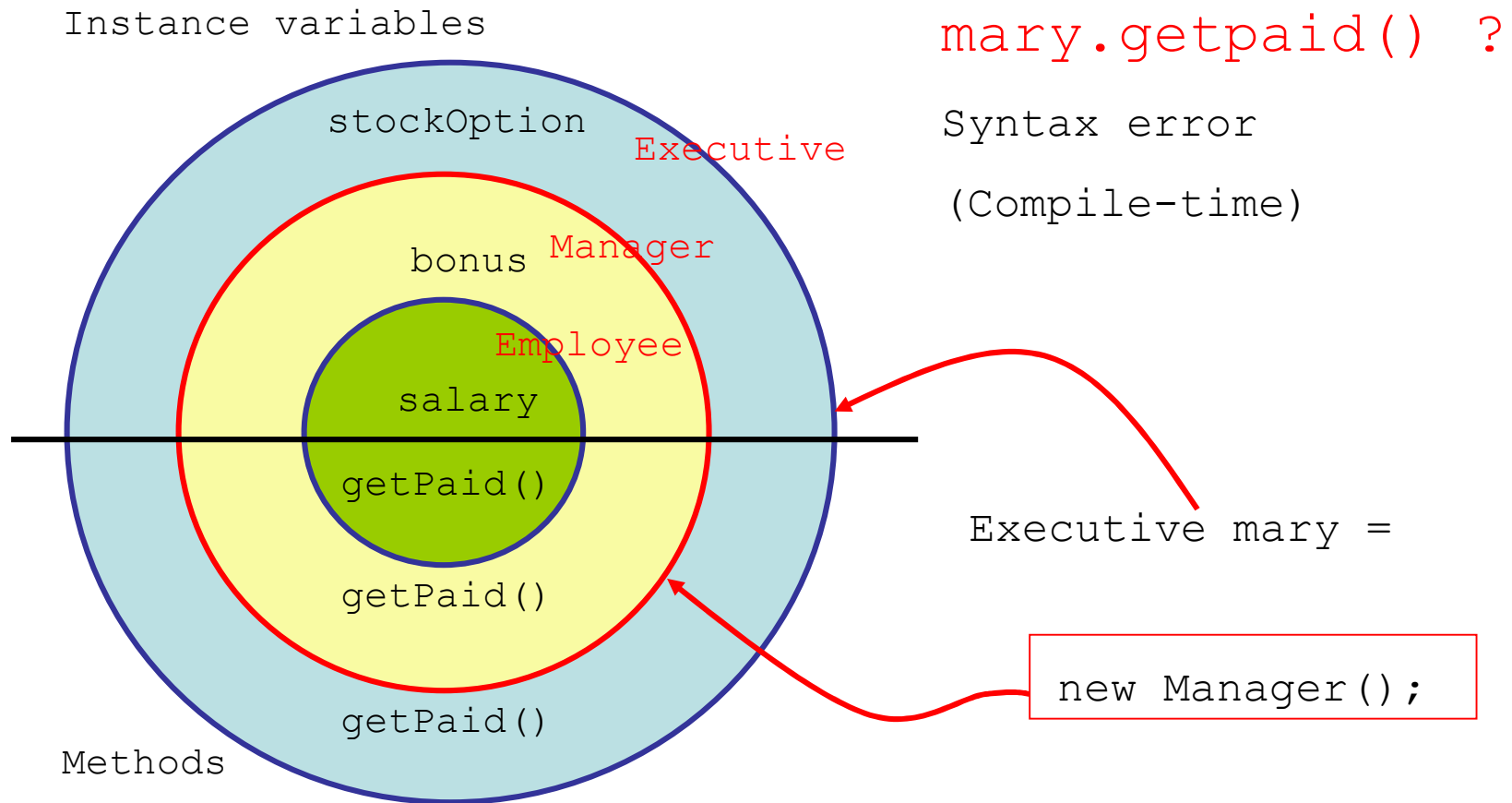
Instance variables

david.getpaid() ?





Polymorphism



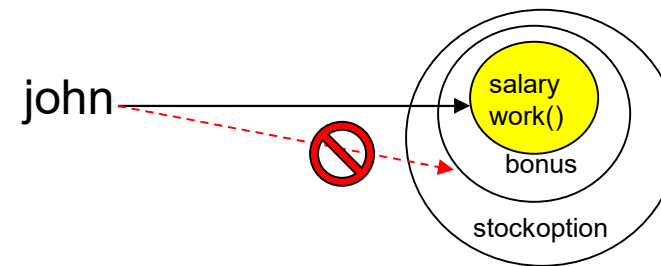


Polymorphism

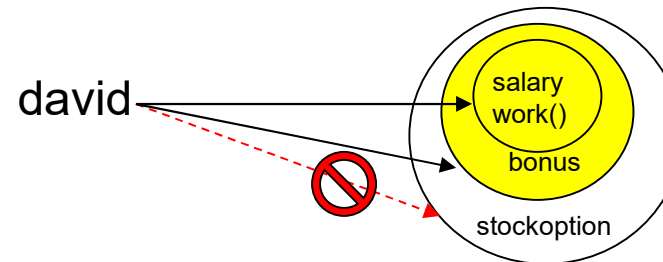
Reference type

“Actual” class type

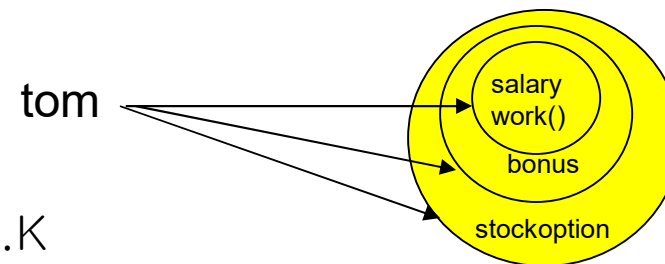
```
Employee john = new Employee();  
john.salary = 100; // O.K  
john.bonus = 10; // Error!
```



```
Employee david = new Manager();  
david.salary = 100; // O.K  
david.bonus = 10; // Error!  
((Manager)david).bonus = 10; // O.K
```



```
Employee tom = new Executive();  
tom.salary = 100; // O.K  
tom.stockoption = 10; // Error!  
((Executive)tom).stockoption = 5; // O.K
```





Polymorphism Example

```
class Employee {  
    int salary = 1000;  
    int getPaid() {  
        System.out.println("Employee: Salary $1,000");  
        return salary;  
    }  
}  
  
class Manager extends Employee {  
    int bonus = 10000;  
    int getPaid() {  
        System.out.println("Manager: Bonus $10,000");  
        return salary + bonus;  
    }  
}  
  
class Executive extends Manager {  
    int stockOption = 100000;  
    int getPaid() {  
        System.out.println("Executive: Stock Option $100,000");  
        return salary + bonus + stockOption;  
    }  
}
```



Polymorphism Example

```
public class PayDemo {  
    static void main(String[] args) {  
        Employee john, mary, david;  
        john = new Employee();  
        mary = new Manager();  
        david = new Executive();  
        System.out.println ("John gets paid $" + john.getPaid());  
        System.out.println ("Mary gets paid $" + mary.getPaid());  
        System.out.println ("David gets paid $" + david.getPaid());  
    }  
}
```



Polymorphism Example

JBuilder - C:/Documents and Settings/selab/jbproject/PayDemo/src/paydemo/PayDemo.java

File Edit Search View Project Run Wizards Tools Window Help

Employee Executive Manager PayDemo

PayDemo.jpx
Employee.java
Executive.java
Manager.java
PayDemo.java

Imports
PayDemo

```
public class PayDemo {  
    static void main(String[] args) {  
        Employee john, mary, david;  
        john = new Employee();  
        mary = new Manager();  
        david = new Executive();  
        System.out.println ("John gets paid $" + john.getPaid());  
        System.out.println ("Mary gets paid $" + mary.getPaid());  
        System.out.println ("David gets paid $" + david.getPaid());  
    }  
}
```

PayDemo.java

Source Design Bean Doc History

C:\JBuilder6\jdk1.3.1\bin\javaw -classpath "C:\Documents and Settings\sela\b\jbproject\PayDemo\classes;C:\JBuilder6\jdk1.3.1\demo\jfc\Java2D\Java2Demo.jar;C:\JBuilder6\jdk1.3.1\jre\lib\i18n.jar;C:\JBuilder6\jdk1.3.1\jre\lib\sunrsasign.jar;C:\JBuilder6\jdk1.3.1\lib\dt.jar;C:\JBuilder6\jdk1.3.1\lib\htmlconverter.jar;C:\JBuilder6\jdk1.3.1\lib\tools.jar" PayDemo.java

Employee: Salary \$1,000
John gets paid \$1000
Manager: Bonus \$10,000
Mary gets paid \$11000
Executive: Stock Option \$100,000
David gets paid \$111000



explain proper and desirable use of polymorphism, especially overriding, in java programs. using examples

Proper and Desirable Use of Polymorphism (Overriding) in Java Programs

Polymorphism, especially **method overriding**, is a fundamental concept in Java that enables **flexibility, maintainability, and scalability** in object-oriented programs. It allows a subclass to provide a **specific implementation** of a method that is already defined in its superclass.

◆ Why Use Overriding?

✓ 1. Enables Dynamic Method Dispatch (Runtime Polymorphism)

- Allows calling overridden methods **based on the actual object type**, not the reference type.
- Promotes **loose coupling** by enabling programs to work with **generic types** but execute **specific behaviors**.

✓ 2. Enhances Code Maintainability & Extensibility

- Avoids **hardcoding logic** for different classes.
- Makes code easier to **extend** without modifying existing logic.

✓ 3. Supports the Open-Closed Principle (OCP)

- Encourages designing systems where **new functionality** can be added with **minimal changes** to existing code.



```
public class Company {
    public static void main(String[] args) {
        Employee emp0 = new Employee("Alex", 100000);
        Employee emp1 = new Manager("Bill", 100000);
        Employee emp2 = new Developer("Carl", 100000);

        emp0.calculateBonus(); // Calls Employee's calculateBonus()
        emp1.calculateBonus(); // Calls Manager's calculateBonus()
        emp2.calculateBonus(); // Calls Developer's calculateBonus()
    }
}

class Employee {
    String name;
    double salary;

    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    void calculateBonus() {
        System.out.println("Bonus for Employee: " + (salary * 0.05));
    }
}
```



```
class Manager extends Employee {
    Manager(String name, double salary) {
        super(name, salary);
    }

    @Override
    void calculateBonus() {
        System.out.println("Bonus for Manager: " + (salary * 0.10));
    }
}

class Developer extends Employee {
    Developer(String name, double salary) {
        super(name, salary);
    }

    @Override
    void calculateBonus() {
        System.out.println("Bonus for Developer: " + (salary * 0.15));
    }
}
```

Output

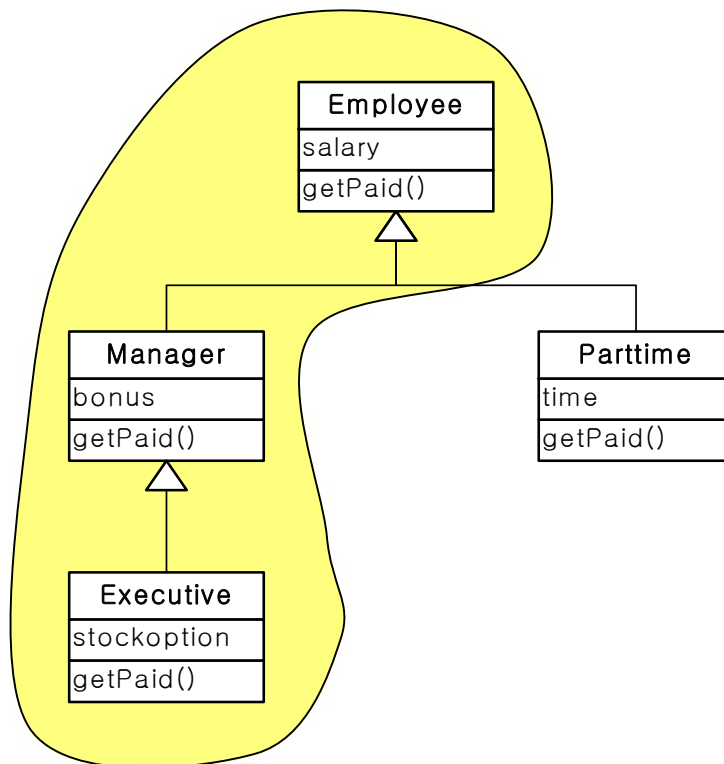
Bonus for Employee: 5000.0
Bonus for Manager: 10000.0
Bonus for Developer: 15000.0

=== Code Execution Successful ===



Difference?

- “Child class” has additional attributes (e.g., bonus, stockoption)



- “Child class” has the same attribute (e.g., salary)
 - Overriding was applied only on calculateBonus method

```
class Manager extends Employee {
    Manager(String name, double salary) {
        super(name, salary);
    }

    @Override
    void calculateBonus() {
        System.out.println("Bonus for Manager: " + (salary * 0.10));
    }
}

class Developer extends Employee {
    Developer(String name, double salary) {
        super(name, salary);
    }

    @Override
    void calculateBonus() {
        System.out.println("Bonus for Developer: " + (salary * 0.15));
    }
}
```



◆ Best Practices for Using Overriding in Java

1. Use `@Override` Annotation

- Helps catch errors if the method signature doesn't match the superclass method.

```
@Override  
void calculateBonus() { ... }
```

2. Use `super` to Call the Parent Method (If Needed)

- Example: A subclass modifies but still calls the parent method.

```
class Developer extends Employee {  
    @Override  
    void calculateBonus() {  
        super.calculateBonus(); // Call superclass method  
        System.out.println("Additional Developer bonus!");  
    }  
}
```



3. Avoid Overriding Static Methods

- Static methods are hidden, **not overridden**.

```
class Parent {  
    static void show() { System.out.println("Parent show"); }  
}  
  
class Child extends Parent {  
    static void show() { System.out.println("Child show"); }  
}
```

4. Use `abstract` Methods When Necessary

- If all subclasses **must** override a method, declare it **abstract** in the parent class.

```
abstract class Employee {  
    abstract void calculateBonus();  
}
```

5. Keep the Overriding Method Signature Identical

- The method name, parameters, and return type should match.



what do you mean by "Static methods are hidden, not overridden"?

What Does "Static Methods Are Hidden, Not Overridden" Mean?

In Java, **static methods belong to the class, not to instances (objects)**. When a subclass defines a static method with the same name as a static method in its superclass, **it does not override the method**; instead, it **hides** it.

◆ Difference Between Overriding and Hiding

Feature	Method Overriding (Instance Methods)	Method Hiding (Static Methods)
Method Type	Instance (non-static)	Static
Binding Time	Runtime (Dynamic Binding)	Compile-time (Static Binding)
Behavior	Method call depends on the actual object type	Method call depends on reference type
How It Works	Subclass provides a new implementation of an instance method	Subclass defines a static method with the same name as the superclass method



◆ Example of Method Overriding (Instance Methods)

Here, the method is overridden, and Java uses **dynamic method dispatch** (runtime polymorphism).

```
class Parent {  
    void display() { // Instance method  
        System.out.println("Parent display()");  
    }  
}  
  
class Child extends Parent {  
    @Override  
    void display() {  
        System.out.println("Child display()");  
    }  
}  
  
public class Test {  
    public static void main(String[] args) {  
        Parent obj = new Child(); // Upcasting  
        obj.display(); // Calls Child's display() (Overriding)  
    }  
}
```

Output:

```
Child display()
```




◆ Example of Method Hiding (Static Methods)

If we make the method **static**, the behavior changes.

```
class Parent {
    static void display() { // Static method
        System.out.println("Parent static display()");
    }
}

class Child extends Parent {
    static void display() { // Static method with the same name (Hiding, not Overriding)
        System.out.println("Child static display()");
    }
}

public class Test {
    public static void main(String[] args) {
        Parent obj = new Child(); // Upcasting
        obj.display(); // Calls Parent's display() (Method Hiding)
    }
}
```

Output:

```
Parent static display()
```



Upcasting?

what's upcasting in java polymorphism?

◆ Upcasting in Java Polymorphism

Upcasting is the process of converting a **subclass reference** into a **superclass reference**. It is an **implicit type conversion** where an object of a subclass is treated as an object of its superclass.

◆ How Upcasting Works

When a subclass object is assigned to a superclass reference, **only the superclass methods and fields are accessible**, even though the actual object is of the subclass type.



```
class Animal {
    void makeSound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void makeSound() {
        System.out.println("Dog barks");
    }

    void fetch() {
        System.out.println("Dog fetches the ball");
    }
}

public class TestUpcasting {
    public static void main(String[] args) {
        Animal myAnimal = new Dog(); // Upcasting (Implicit)
        myAnimal.makeSound(); // Calls Dog's overridden method

        // myAnimal.fetch(); // ✗ Compilation error (not accessible)
    }
}
```



My Recommendation on Java Polymorphism

- **Use it only when its use simplifies code complexity and readability**
 - Too many layers of inheritance (Deep Hierarchies)
 - Overriding Methods with Unexpected Behavior
 - Overriding with Too Many Indirections (Difficult Debugging)
 - Overriding in an Unrelated Context
 - Using Overriding in Utility Methods (Misleading Code)

what are examples of java polymorphism and overriding where code becomes less readable?

♦ Examples Where Java Polymorphism & Overriding Reduce Readability

While polymorphism and method overriding **improve flexibility and maintainability**, they can also lead to **confusion and poor readability** if misused. Below are some scenarios where polymorphism and overriding make the code **harder to understand, debug, or maintain**.



Polymorphism Bad Practices

◆ Key Takeaways

Bad Practice	Why It Reduces Readability
Deep Inheritance Hierarchy	Hard to track method calls
Unexpected Overriding Behavior	Breaks user expectations
Too Many Indirections	Debugging becomes difficult
Overriding in an Unrelated Context	Inheritance is misused
Misleading Overriding in Utility Methods	Code meaning is lost

✅ Best Practices to Avoid Readability Issues

- ✓ Use clear and predictable method overriding
- ✓ Follow the Liskov Substitution Principle (LSP)
- ✓ Prefer composition over deep inheritance
- ✓ Don't override methods unnecessarily
- ✓ Keep method behavior consistent with its name